Step 2: Software Development Project

Week 1 — Implementation Foundation and Project Structure

This report documents the first working implementation of the Academic Research Paper Assistant. The Week 1 goal was to create an organized application structure and demonstrate that a user can upload an academic PDF and extract its selectable text.

1. Establish the application source tree

The current source tree is organized as follows:

```
FA26_Mistry_Monalben_COMP_699_A/
├── .gitignore
├── pyproject.toml
├── uv.lock
├── README.md
├── data/
│   ├── chroma/
│   └── papers/
├── docs/
│   ├── Step_1/
│   └── Step_2/
└── src/
    ├── app.py
    ├── generation/
    ├── ingestion/
    │   └── pdf_loader.py
    └── retrieval/
```

The main components are:

- The project separates application code, data, and documentation.
- **Src/app.py** contains the Streamlit user interface.
- **src/ingestion/pdf_loader.py** reads an uploaded PDF and extracts text from its pages.

Step 2: Software Development Project

- **pyproject.toml** defines the project dependencies.
- **uv.lock** records resolved dependency versions.
- **data/papers/ folder** is reserved for research PDFs.
- **data/chroma/ folder** is reserved for local vector storage.
- **The retrieval/ and generation/ folders** are planned components; their RAG functionality is not implemented in Week 1.

2. Configure the development environment

Technology	Purpose in This Project
Python	Application logic and PDF processing.
Streamlit	Web interface for uploading PDFs and viewing extracted text.
pypdf	Reads PDF pages and extracts selectable text.
uv	Installs and manages the Python environment and dependencies.
LangChain / Ollama	Selected for later RAG and local language model work. A separate basic connectivity test was completed earlier; these are not yet integrated into the current Streamlit workflow.
ChromaDB	Planned local vector store for document-chunk embeddings and retrieval.

3. Installing Dependencies and Running the Application

Prerequisites

Install Python 3.13 or a compatible Python version, uv, and Git.

Setup from the repository

Run the following commands in a terminal:

```
git clone --branch project/rag-poc
https://github.com/monalmistry-
web/FA26_Mistry_Monalben_COMP_699_A.git

cd FA26_Mistry_Monalben_COMP_699_A
```

```
uv sync
```

```
uv run streamlit run src/app.py
```

After running the application command, Streamlit displays a local URL in the terminal. Open that URL in a web browser, choose a valid PDF using the upload control, and review the page count and extract text.

To stop the application, press **Ctrl+C** in the terminal.

4. Summary of What Currently Executes Successfully

The following functionality has been implemented and tested:

- The Streamlit application starts locally.
- A user can upload a PDF file through the interface.
- The application reads the PDF and displays its page count.
- Selectable text is extracted from the PDF and displayed in the interface.
- A warning appears when no selectable text is found, which may occur with scanned PDFs.
- The upload control accepts PDF files and rejects a .txt file.

A research PDF was successfully uploaded during testing. Its page count and extracted text were displayed.

One limitation observed during testing is that some PDF text contains line-break hyphenation or unusual spacing. Scanned PDFs may require Optical Character Recognition (OCR), which is not implemented in the current version.

The current application does not yet provide document chunking, embedding generation, vector database operations, question answering, or source/page citations.

5. Design Decisions and Changes

Streamlit was selected instead of the initially considered React frontend and FastAPI backend to simplify development and testing. The initial implementation focuses on PDF upload and text extraction using pypdf.

Step 2: Software Development Project

LangChain, Ollama, and ChromaDB are planned for the future RAG workflow but have not yet been integrated. The design prioritizes simplicity, local processing, and reduced reliance on paid external APIs.

6. Planned Week 2 Functionality

The next milestone will focus on implementing selected core RAG functionality:

1. Split extracted PDF text into smaller chunks.
2. Generate embeddings using a compatible embedding model.
3. Store document chunks and embeddings in ChromaDB.
4. Implement similarity search to retrieve relevant passages for a user's question.
5. Add basic tests for PDF extraction, chunk creation, and retrieval behavior.

The embedding model and integration details will be verified during implementation. Local LLM answer generation and the integrated end-to-end workflow will follow in later development.

Conclusion

Week 1 established a working foundation for the Academic Research Paper Assistant. The application launches successfully, accepts PDF uploads, counts pages, and extracts selectable text. The next stage will focus on preparing extracted text for semantic retrieval and implementing the components required for question answering.